RSolve Manual

Jonathan Michel

October 30, 2026

Contents

1 Introduction **1**

2 Installation **2**

3 Constructing Networks with `hnet_maker` **2**
3.1 Overview 2
3.2 Network Construction Process 3
3.3 Alternate Output Forms 4
3.4 Auxilliary Information 4
3.5 Interactive Mode 4
3.6 Automated Mode 5

4 Solving for Potentials and Resistances **9**

1 Introduction

This manual describes RSolve, a suite of software to create networks and evaluate their electrical resistances. Networks are lattice-based, and can be hierarchical, in the sense that, when a network is constructed, each of its bonds can be replaced by a finer-scale network. This process can, in principle, be repeated *ad infinitum*. At each length scale, disorder can be added in two ways: by randomly removing a subset of connections in the network, and by adding position disorder to the vertices of the network.

Once a netwok is created, it can be reported in three distinct ways: as a planar line graph, as an SVG file, and as an extruded surface stored in an STL file. The last two options were designed to allow the creation of physical specimens for experimental work. STL files can also be imported by software packages that can be used to create FEM meshes to allow finite element simulation of current flow. This method differs from that used by the program `rsolve`, described below.

This project originated with graduate work at Georgia Tech in the lab of Professor Peter Yunker. Professor Yunker supplied the original idea for the project

and supported the development of the initial version of this software. I have also benefitted from discussions with Henry Murphy, whose testing of the software's capabilities highlighted numerous opportunities for improvement. This software has benefitted from the open source software packages SuiteSparse, by Timothal Alden Davis, and Triangle, by Jonathan Richard Shewchuk.

2 Installation

This software is written mostly in standard C++. It depends on the C-language library Triangle, by Jonathan Richard Shewchuk, as well as Shewchuk's geometric predicates library. These need not be installed by the user; rather, lightly modified versions of the source code for these software packages are included with the RSolve source code. The RSolve library also requires SuiteSparse, a library for solving large linear systems involing sparse matrices, such as those that are typically encountered when studying networks of resistors. SuiteSparse, in turn, requires efficient BLAS and LAPACK libraries. I have personally linked SuiteSparse against the Intel MKL implementations of these libraries. Choosing efficient BLAS and LAPACK libraries is *extremely* important to ensure good performance from this software. Certain SuiteSparse capabilities require OpenMP to be installed as well, and some SuiteSparse components can be further expedited by CUDA.

Apart from the specialized dependencies mentioned above, this software requires a modern C and C++ compiler, as well as GNU Make. Once all dependencies are installed, building the library should be as simple as typing "`make`". This should build a subdirectory called `lib`, as well as the executables `hnet_maker` and `rsolve`.

3 Constructing Networks with `hnet_maker`

3.1 Overview

The program `hnet_maker` operates in two modes: interactive and automated. In the interactive mode, the program prompts the user for each stage of the network creation process. In the automated mode, an input file is passed to the program using the syntax `-j file_name`. Information is parsed from the job file and used to automatically generate one more more networks. While the interactive mode may be useful for tinkering while designing a parameter sweep, the automated mode is likely much more useful for production runs. In the next subsection, the network creation process is described in detail. In the following subsections, specific explanations of the interactive and automated modes are provided.

3.2 Network Construction Process

Whatever the mode of operation, network creation is organized into levels. At each level, a network structure is specified in terms of a lattice and a scale factor that sets the lattice spacing. Lattices are built row-by-row within a set of prescribed Cartesian bounds. Lattice information is read from .lat files, which specify the locations of vertices and the displacements from vertices to neighbors to which those vertices are connected by a bond. Disorder can be applied to networks in two ways: vertices of the network can be randomly displaced according to a Gaussian distribution, and connections between vertices can be removed, such that the of proportion of retained vertices is p, with $p \in (0, 1]$. Dilution of bonds can be carried out either by ensuring all vertices belong to a single connected component, or by a purely random process. Purely random dilution can be specified by the `-d` flag.

After the first network structure has been applied, an arbitrary number of additional levels of structure can be specified. Note that levels should be specified in order of *increasing* length scale. At each level, the user must specify the width of the bonds. For all but the smallest length scale, the user *must* specify a width greater than zero. Also note that, for good results, lattices across scales should generally have structures and lattice constants such that they are commensurate - that is, large-scale vertices coincide with small-scale vertices in a regular pattern. The large-scale network structure may also be disordered by displacing vertices and removing bonds.

If position disorder is added to large-scale vertices in a hierarchical network, the user may specify that a grain-based approach should be used. This means that as much of each large-scale edge as possible will be filled with a small-scale lattice with a specified structure. At boundaries between grains constructed in this manner, edges will be formed connecting vertices within some cutoff distance such that no two edges intersect. If two prospective edges would intersect, the shorter of the two edges will be chosen.

Once all levels of strcuture are specified, network creation proceeds in the opposite order of that in which levels are specified. For hierarchical network creation, once a network is generated at a given length scale, each bond is given a non-zero width to generate an envelope. The network one level below this network in the hierarchy is then created, and only bonds falling within the large-scale envelope are kept. When small-scale bonds are diluted in a hierarchical network, they are removed in such a way that the large-scale bond topology is preserved.

If vertices at any length scale are displaced to create site disorder, there will, in general, be no clearly defined top and bottom edges of the network. This will make it challenging to define well-posed boundary conditions to simulate the resistance of a network. To address this issue, the user may specify that the network should be prepared for grips. If this option is selected, the user must specify lower and upper y coordinates such that bonds protruding below or above these bounds will be truncated at the prescribed heights.

3.3 Alternate Output Forms

By default, once a network has been constructed, it is reported in a file format that first specifies the number of vertices and edges, then lists all vertices, and finally specifies edges as pairs of integer indices into the list of vertices. While this offers a concise description of the network, I have found it useful to report networks by specifying networks by enumerating edges only. In this more verbose format, each edge is specified by three lines. The first two lines give the Cartesian components of the endpoints of the edge, and the third line is blank. This output mode can be selected using the "`-v`" switch. Note that this verbose format is appropriate only for quick visualization – the program `rsolve` expects networks to be reported in the concise format.

In addition to specifying networks as planar line graphs, i.e. a set of vertices, and edges connecting those vertices, there are two modes to enable the fabrication of physical samples. SVG files can be created by passing the "`-s`" switch. If this mode is used, the network at the smallest scale should have bonds of non-zero width. This mode is intended to allow the creation of samples that can be made with a laser cutter or drag knife plotter. In this mode, the user must specify additional information about tabs to append to the top and bottom of the network. Tab outlines have four components: outward-curving quarter ellipse arcs, rounded corners, vertically aligned parallel edges, and horizontally oriented top and bottom edges. The user must specify the width and height of elliptical arcs, the radius of curvature of corners and the heights of the final top and bottom tabs.

STL files may also be generated, and can be used for 3D printing and as a precursor to FEM mesh construction. In this case, networks at the smallest scale also must have a non-zero width. Networks are extruded out of the plane by a user-specified height. The resulting surface is triangulated and written to a binary STL file.

3.4 Auxilliary Information

In addition to the network itself, information can be stored about the network at the largest length scale, the degree to which small-scale bonds are aligned with the large-scale bonds in which they lie, and the polygonal tiling into which the large-scale envelope is partitioned in preparation for selecting small-scale edges. Reporting of the alignment is requested with the "`-a`" switch, and reporting of the large-scale polygonal tiling is requested with the "`-p`" switch.

3.5 Interactive Mode

In the interactive mode, the required information is requested one field at a time through a series of command line prompts. The expected answer may be one or more numbers, a string literal, or, in the case of yes/no questions, either "`y`" of "`n`". While perhaps useful as a preliminary means of exploring different options, this method is not very efficient when preparing for parameter sweeps.

One could create a shell script to automate interaction, but a more elegant alternative is described below.

3.6 Automated Mode

In the automated mode, all necessary information for network construction is specified in a special script file format. This format is organized into hierarchcially arranged *elements*, each of which may have zero or more child elements. An element begins with a name and an opening left curly brace, and ends with a closing right curly brace. Apart from child elements, elements may also have *attributes*. These are specified using the syntax `element_name = "value"`. A value may either be a single entity or multiple entities delimited by spaces. After a name-value pair has been specified, it should be separated from any successive member of an element by a comma. In addition to the specification of elements and attributes, the user may also include blank lines and comment lines. A comment line is distinguished by the property that its first non-whitespace character is a pound sign. These characteristics collectively define a serialization scheme I have named ISLE (Input Serialization LanguagE) [1]. By convention, I give these files the extension .isle.

In the context of `hnet_maker`, the outermost element of an ISLE file should be "`hnet_job`". This element must have as children one or more elements of type "`layer`". The outer `hnet_job` element should have at least the attributes `bounds` and `output_name`. The value of the `bounds` attrtibute should be a list of four numbers, giving the minimum x and y coordinates and maximum x and y coordinates of the bounding box containing the network. The output name should be a string, to which additional information and a file extension will be appended to create the file name for exporting a newly created network.

The `hnet_job` element supports numerous optional attributes. If site disorder is added to vertices on any length scale, it will likely be desirable to specify heights at which the network should be truncated to create clean top and bottom boundary conditions. This can be accomplished by setting the attibute `add_grips` to "true". One must also set the attribute `grip_heights` in this case, with a value of a top and bottom truncation height, separated by whitespace.

Multiple realizations for each unique combination of network construction parameters may also be requested. This is accomplished by setting the attribute `realizations` to an integer greater than 1.

If a range of bond occupations is specified at a given construction stage (see below), multiple different combinations of network parameters will be created. In this case, numerical labels will be appended to the base output name to indicate the bond occupation. By default, three decimal digits will be used. This value can be changed by setting the attribute `report_digits`.

If a network is to be deluted, by default, dilution begins with the construction of a random minimum spanning tree. If completely random bond removal

[1] It may seem odd to design one's own serialization language, but I couldn't find one that suited my criteria.

is preferred, this can be accomplished by setting the attribute `connected` to "false".

If the user wishes to ensure adjacent vertices along the top and bottom of the network are joined by bonds, this can be accomplished by setting the parameter `connect_top_bottom` to "true". This feature may be deprecated in the future.

As noted above, various sorts of auxiliary information apart from the final network can be reported. In the automated mode, these options can be activated by setting the following output file name patterns to a non-empty string:

- `alignment_file`: A record of the degree of alignment of small scale bonds with the large-scale bonds within which they are contained
- `poly_file`: The file containing the polygonal tiles into which the top-level envelope is partitioned
- `skeleton_file`: A file containing the "skeleton" of the top-level envelope, before a non-zero width is imparted to each bond
- `top_file`: A file containing the top-level envelope used to judge whether small-scale bonds should be included

Each `layer` element within the `hnet_job` parent element should have at least the following properties:

- `lattice_file`: The name of a lattice description file from which to read information about network geometry and topology
- `scale`: A scale factor to set the lattice constant at a given length scale
- `width`: The width of each bond in the network
- `bond_occupation`: The fraction of bonds to retain. This quantity should either be a single number between zero and one, inclusive, or a minimum value, maximum value, and step size.

If a range of bond occupations is specified, a job will be created for each bond occupation in this range. If a range of bond occupations is specified for multiple levels, then the total number of networks created will be the product of the numbers of distinct bond occupations at each length scale.

The `layer` element also has a number of optional attributes. If the user wishes to apply Gaussian random noise the vertex positions, this can be specified with the field `displacement`, which gives the standard deviation of this noise. For any level of structure below one for which position disorder is added, the corresponding `layer` element can specify that grain-based construction should be used by setting the attribute `grain_based` to true. In this case, the parameter `stitch_cutoff`, giving the greatest length of a bond stitching two grains, should also be specified.

To make the above concepts more concrete, two sample job files are provided below. The first demonstrates the creation of a small hierarchical network to be reported to a file as a planar line graph.

```
hnet_job {

   output_name = "tri_tri_2_2_1_p1",
   bounds = "0 0 2 2",

   #The structure at the small length scale
   layer {
      lattice_file= "tri.lat",
      scale = ".1",
      width = "0",
      bond_occupation = "1",
   }

   #The structure at the large length scale
   layer {
      lattice_file= "tri.lat",
      scale = "1",
      width = ".2",
      bond_occupation = "1",
   }
}
```

Figure 1: This ISLE file generates a hierarhical triangular lattice in which large-scale bonds are 10 times the length of small-scale bonds.

The ISLE file in Figure 3 demonstrates the creation of SVG and STL files. The attributes for both types of output are included. One can choose which sort of output is actually created from the command line, by supplying either the "`-s`" switch for SVG output or the "`-e`" switch for output as an extruded surface in the STL format. Unneeded attributes will simply be ignored, such that a single job file can do double duty. The results of these two usage modes are displayed in Figure 4.

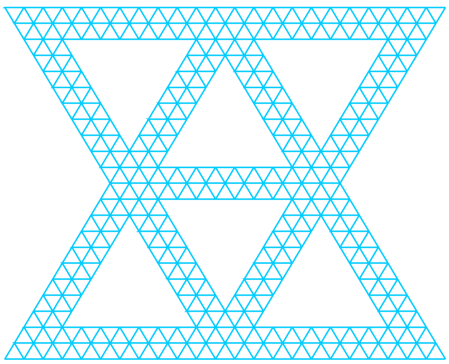


Figure 2: A small, hierarchical triangular lattice created by the script in Figure 1.

```
hnet_job {

   output_name = "tri_tri_2_2_1_p1",
   bounds = "0 0 2 2",
   extrusion_height = "0.02",
   taper_width = ".25",
   taper_height = ".5",
   corner_radius = ".1",
   tab_height = ".5",

   #The structure at the small length scale
   layer {
      lattice_file= "tri.lat",
      scale = ".1",
      width = "0.02",
      bond_occupation = "1",
   }

   #The structure at the large length scale
   layer {
      lattice_file= "tri.lat",
      scale = "1",
      width = ".2",
      bond_occupation = "1",
   }
}
```

Figure 3: This ISLE file generates a hierarhical triangular lattice in which large-scale bonds are 10 times the length of small-scale bonds, but differs from the example above in that small-scale bonds have non-zero width, and attributes are specified to support SVG and STL file creation.

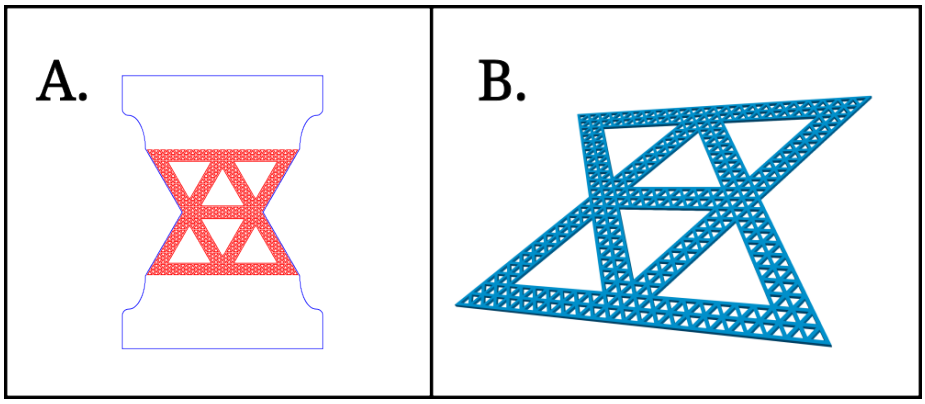


Figure 4: The results of running `hnet_maker` with the job file in Figure 3 are shown. Panel **A** shows the output produced by the "`-s`" switch, and panel **B** shows the output produced by the "`-e`" switch.

4 Solving for Potentials and Resistances

Interaction with `rsolve` is fairly simple. The user can either supply the name of a network description file at the command line using the switch `-f`, or run the program in interactive mode, in which case the program will prompt the user for a file. If the user wishes for the potential to be written to a file, this can be indicated by the `-p` flag, which takes as an argument the name of the file to which the potential of each node should be written. In interactive mode, triggered by running the program with no input, the user is asked if a potential file is desired, and upon answering in the affirmative, the user is asked for the name of this file.

The simulation carried out by `rsolve` is one in which unit potential difference is applied to the network. At the moment, the program is designed with the assumption that each resistor is of the same resistance. In the future, this assumption may be relaxed to allow for the study of heterogeneous resistor networks.

To compute the resistance of a network, the potential is required. Any node in the network not at either the top or bottom is required to have no net current passing into or out of it. Nodes at the top and bottom are required to have prescribed electric potentials. These conditions collectively can be used to define a matrix $\mathbf{A}$. Let the vector $\vec{\varphi} \in \mathbb{R}^{1\times N}$ be a list of potentials of all nodes, where N is the number of nodes in the network. If a node is on the interior of the network, the steady state condition dictates that the total current flowing in or out of the node is zero. The current flowing to node i from node j is

$$\phi_{ij} = \frac{\phi_j - \phi_i}{R_{ij}}, \tag{1}$$

where R_{ij} denotes the the resistance of the resistor connecting these two nodes. Let $nn(i)$ denote the set of all nodes j connected to node i by a resistor. If node i is an interor node, the ith row of $\mathbf{A}$ is then defined by

$$a_{ij} = \begin{cases} \frac{1}{R_{ij}}, & j \in nn(i) \\ -\sum_{k\in nn(i)} \frac{1}{R_{ik}}, & j = i \\ 0, & \text{otherwise,} \end{cases} \tag{2}$$

where a_{ij} denotes the jth component of the ith row of $\mathbf{A}$. If the node i is on the top or bottom of the network, the corresponding row of $\mathbf{A}$ has a simpler form:

$$a_{ij} = \delta_{ij}, \tag{3}$$

where δ denotes the Kronecker delta function.

The potential is then obtained by solving the equation

$$\mathbf{A}\vec{\varphi} = \vec{b}, \tag{4}$$

where $\vec{b} \in \mathbb{R}^{1\times N}$ is defined by

$$b_i = \begin{cases} 1, & i \text{ is a top vertex} \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Equation (4) is solved by `rsolve` using the sparse QR decomposition package of the SuiteSparse library.

With the solution for the potential in hand, the resistance is computed by dividing the potential difference across the network by the current flowing through the network. Denoting the set of top vertices by $\mathcal{T}$, the current flowing into the top vertices of the network, I_{top}, is given by

$$I_{top} = \sum_{i \in \mathcal{T}} \sum_{j \in nn(i)} \frac{\phi_j - \phi_i}{R_{ij}}. \tag{6}$$

As the simulation assumes a unit potential difference between the top and bottom of the network, the network resistance, R_{net}, is computed as

$$R_{net} = \frac{1}{I_{total}}. \tag{7}$$

References

[1] T. A. Davis. Algorithm 915, Suitesparseqr: Multifrontal multithreaded rank-revealing sparse QR factorization. *ACM Transactions on Mathematical Software*, 38(1):1–22, 2011.

[2] G. Kirchhoff. Über die Auflösung der Gleichungen, auf welche man bei der Untersuchung der linearen Vertheilung galvanischer Ströme geführt wird (on the Solution of the Equations to which one is Led in the Course of Investigating the Linear Distribution of Galvanic Currents). *Annalen der Physik und Chemie*, 72(12):497–508, 1847.

[3] S. Kirkpatrick. Percolation and Conductivity. *Reviews of Modern Physics*, 45(4):574–588, 1973.

[4] S. Redner. Fractal and Multifractal Scaling of Electrical Conduction in Random Resistor Networks. In R. A. Meyers, editor, *Mathematics of Complexity and Dynamical Systems*, pages 446–462. Springer, New York, NY, 2012.

[5] Jonathan Richard Shewchuk. Triangle: Engineering a 2d quality mesh generator and Delaunay triangulator. In Ming C. Lin and Dinesh Manocha, editors, *Applied Computational Geometry Towards Geometric Engineering*, pages 203–222, Berlin, Heidelberg, 1996. Springer.

[6] Jonathan Richard Shewchuk. Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates. *Discrete & Computational Geometry*, 18:305–363, 1997.